

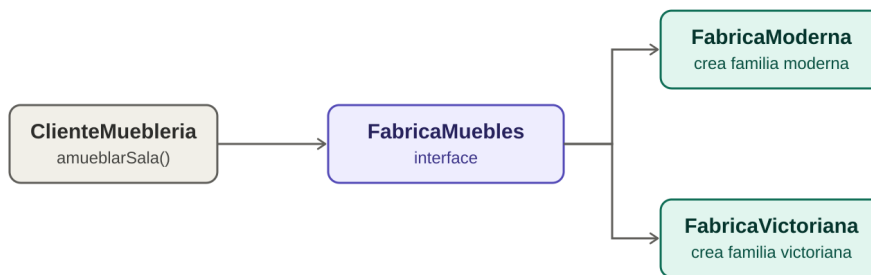


Materia: Programación III – 3º Trabajo Práctico.

Nombre y Apellido del Alumno: Montoya Gonzalo Alejo – CI180156

El presente trabajo práctico presenta 6 patrones de diseño hechos en Java: Abstract Factory, Adapter, Decorator, Facade, Factory Method y Singleton. Para cada uno se explica el problema que se quería resolver, por qué se eligió ese patrón, y abajo se muestra el diagrama de clases.

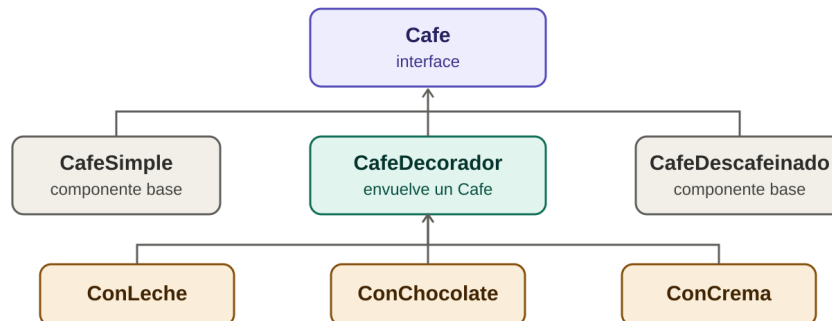
Patrón 1 – Abstract Factory: El problema era una mueblería que vende sillas y mesas en dos estilos, moderno y victoriano, y los muebles de un mismo estilo tienen que combinar entre sí. Se decidió utilizar Abstract Factory porque crea familias de objetos que van juntos: el cliente solo conoce la interfaz *FabricaMuebles* y para cambiar de estilo alcanza con usar otra fábrica. Sin el patrón habría que llenar el código de if y sería fácil mezclar estilos por error.



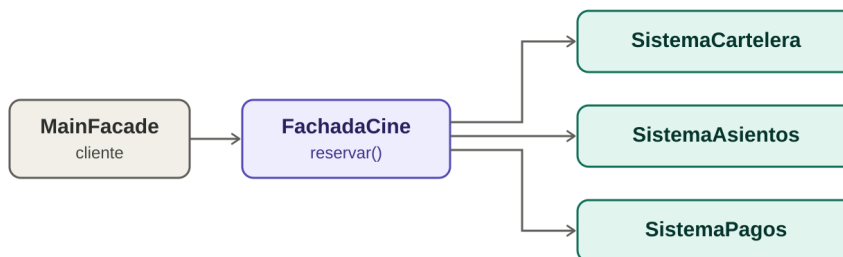
Patrón 2 – Adapter: En este caso el sistema quiere cobrar usando la interfaz *PasarelaPago*, pero ya existía un sistema viejo (*SistemaViejoPago*) con métodos distintos que no se puede modificar. Se optó por Adapter, que hace de traductor: *AdaptadorPago* implementa la interfaz que espera el cliente y por dentro llama al sistema viejo. Así se reutiliza código que ya funciona sin tocarlo y sin cambiar *ClientePago*.



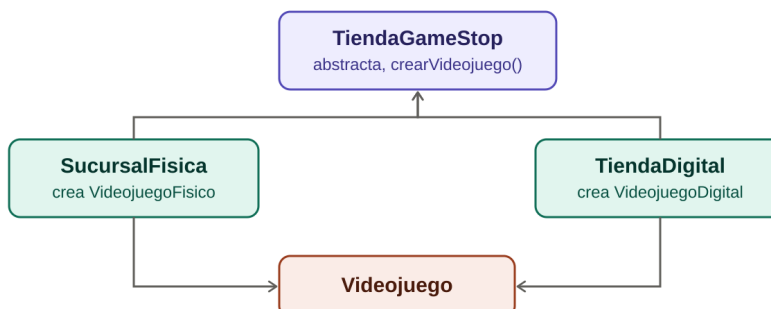
Patrón 3 – Decorator: El problema consistía en un café que puede llevar leche, chocolate, crema o cualquier mezcla, donde cada extra cambia la descripción y el precio. Hacer una clase por cada combinación llevaría a tener demasiadas clases. Con Decorator se envuelve el objeto en vez de usar herencia: `new ConCrema(new ConLeche(new CafeSimple()))` arma cualquier combinación con solo 3 decoradores, y si aparece un ingrediente nuevo es una sola clase más.



Patrón 4 – Facade: Para reservar una entrada de cine hay que usar varios sistemas: la cartelera, los asientos y el pago. Si el cliente los maneja uno por uno tiene que saber cómo funciona cada uno y en qué orden llamarlos. Se utilizó Facade porque FachadaCine ofrece una interfaz simple y coordina todo por dentro: el cliente hace una sola llamada y, si un subsistema cambia, solo se arregla la fachada.



Patrón 5 – Factory Method: GameStop vende juegos físicos y digitales. Cada tipo se crea distinto, pero para el resto del programa todos son un Videojuego. Se eligió Factory Method porque deja que las subclases decidan qué objeto crear: TiendaGameStop define crearVideojuego y SucursalFisica y TiendaDigital devuelven cada una su tipo de juego. Así el código no depende de un new concreto y agregar un formato nuevo es solo agregar una subclase.



Patrón 6 – Singleton: Varios módulos (*ModuloInterfaz* y *ModuloNegocio*) necesitan leer la misma configuración. Si cada uno creara la suya podrían quedar con valores distintos y el programa se contradiría. Se decidió aplicar Singleton para garantizar una sola instancia de *ConfiguracionGlobal* con un único punto de acceso, *obtenerInstancia()*. Así todos ven los mismos datos y no se duplican objetos en memoria.

